

Real-Time Operating Systems

GNC Software Onboarding Lecture #3

What is an Operating System?

- ▶ MacOS, Windows, Linux
- ▶ An **operating system** (OS) is software that acts as an intermediate layer between user programs and the computer's hardware
- ▶ Responsibilities
 - ▶ Allocating CPU time between user programs
 - ▶ Delegating memory to each program
 - ▶ Interacting with the hardware on the computer - memory, storage, USB devices, displays
 - ▶ The OS will allow user programs to indirectly control this hardware

What Does “Real-Time” Mean?

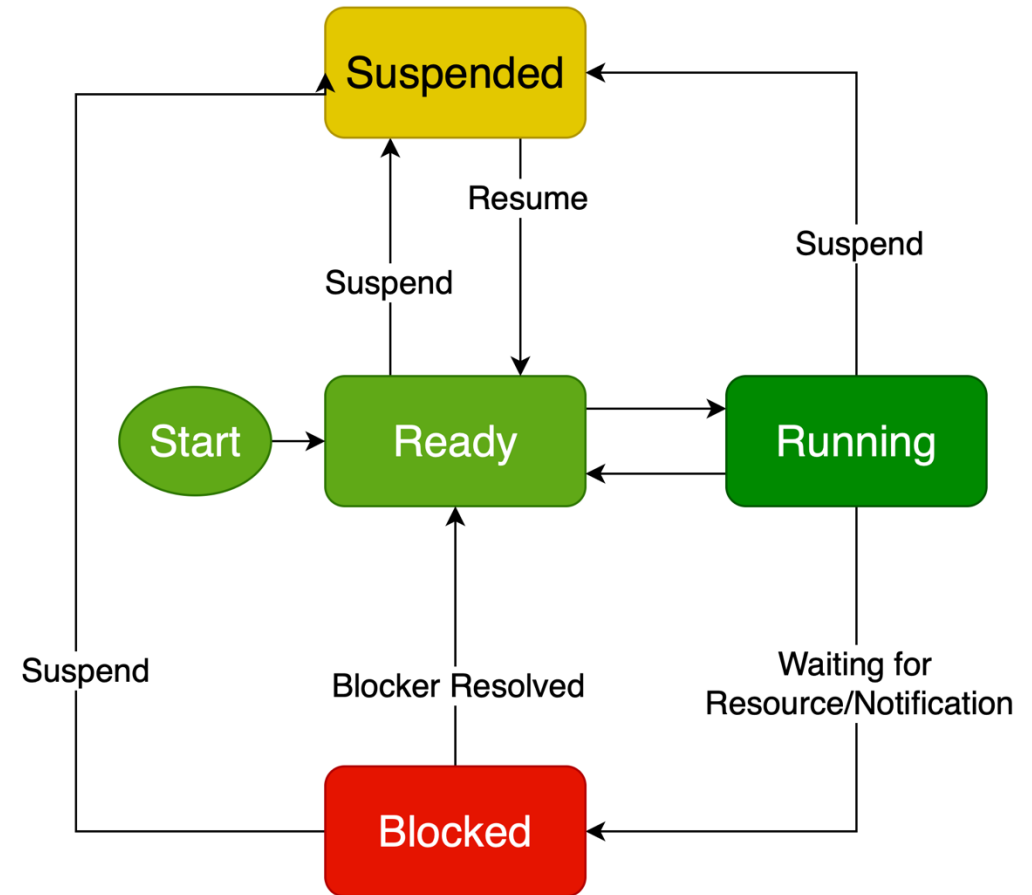
- ▶ On a desktop environment, operating systems may go several milliseconds between executing tasks
 - ▶ Usually, this delay is still not noticeable to users
 - ▶ Also caused by background processes
- ▶ This is acceptable for systems in which it is not strictly necessary that things happen exactly when they need to
 - ▶ If you click your mouse and there is a few millisecond delay, it's not the end of the world
- ▶ In a rocket, what if the OS takes a few milliseconds longer to get data from a sensor?
 - ▶ Now the control loop is lagging behind the true state of the rocket, and we all of a sudden run into some pretty serious problems
- ▶ In rocket flight software, precise timing is critical

Real-Time Operating Systems

- ▶ A **real-time operating system** (RTOS) is an operating system that specializes in responding quickly to real-time events
- ▶ GNC uses a free and open-source solution developed by Amazon called **FreeRTOS**
- ▶ Components of FreeRTOS
 - ▶ Tasks - independent loops that perform some dedicated function
 - ▶ Notifications - allows tasks to send signals to each other
 - ▶ Stream buffers - allows tasks to send data to each other
 - ▶ Mutexes - guarantees that only one task is accessing a resource at a time

Tasks

- ▶ A task is the basic unit of execution in FreeRTOS
- ▶ They are functions that continuously loop
- ▶ Only one task can truly be running at a time, but the scheduler switches between tasks extremely quickly to give the illusion that they are executing in parallel
- ▶ Tasks have specific states: ready, blocked, running, suspended



Notifications

- ▶ Say that in our flight software, we need to wait for some external event to happen before we begin logging data
 - ▶ Ex. Waiting for a “go” signal from ground station
- ▶ In the task that logs data, wait on a notification from another task that handles telemetry
- ▶ When we receive a notification, check that it is the “go” signal

Stream Buffers

- ▶ Say we have task A that produces some data such as sensor readings, and we want to process that data in task B
 - ▶ This frees task A to read more sensor data while task B is processing
- ▶ We could use a **stream buffer** to send sensor readings from A to B
- ▶ You can think of a stream buffer as a pipe from task A to task B through which task A will send data and task B will receive it
- ▶ Task B will wait for data in the stream buffer and process it

Mutexes

- ▶ Say that instead of sending data from one task to another, we want to share some data between an arbitrary number of tasks
- ▶ We cannot just create a global variable and read/modify it in each task that needs it
 - ▶ This creates undefined behavior, as we may switch to a different task in the middle of modifying that global variable
- ▶ Instead, one solution is to create a **mutex**
- ▶ A mutex signals access to a shared resource between tasks such that only one task can own that resource at a time
 - ▶ Before modifying/reading a shared variable, you must take ownership of the mutex
 - ▶ It is not guaranteed that the resource will be available, so you can specify a timeout

Practice Exercises

- ▶ Go to <https://github.gatech.edu/gnc/software-onboarding> and do exercise 3
 - ▶ 1. Run “git clone <https://github.gatech.edu/gnc/software-onboarding>”
 - ▶ 2. Create a branch by running “git checkout -b <first name>-exercise3”
 - ▶ Ex. Sam would run “git checkout -b sam-exercise3”
 - ▶ This creates a new branch
 - ▶ 3. Do the exercise following the instructions in the exercise 3 README
 - ▶ 4. Stage your changes by running “git add .”
 - ▶ 5. Commit your changes by running “git commit -m <commit message>”
 - ▶ Ex. git commit -m “Sam exercise 3”
 - ▶ 6. Push your changes by running “git push origin <first name>-exercise3”
 - ▶ 7. Create a pull request in GitHub to merge your branch into the main branch

Reference Materials

Useful Links

- ▶ <https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/00-Overview> - Beginner's guide to FreeRTOS
- ▶ <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/00-Developer-docs> - FreeRTOS documentation
- ▶ <https://github.gatech.edu/gnc/flight-software/tree/refactoring-main/Tasks> - a few FreeRTOS tasks used in the current flight software